

Praktikum 1 — Foundations of Data Science

Data Science • Magister Ilmu Komputer, FMIPA UGM Companion to **Chapter 1** of *Data Science: A Practical Foundation*.

What you will do

1. Build and verify a reproducible Python environment.
2. Learn the reproducibility rules this course enforces in every later week.
3. Take a first look at a real dataset with `pandas`.
4. Meet the base-rate trap — the reason accuracy is a poor default metric.
5. Frame a vague request as a data science problem (CRISP-DM phase 1).

Assessment. This meeting carries no assignment weight, but everything from week 2 onwards assumes the environment built here works.

How the notebooks work

- Cells marked **DEMO** are worked for you. Read them, run them, change them.
- Cells marked **TASK** are yours. Replace `None` with your answer.
- `checks.task("x.y", answer)` tells you whether you are right. It never crashes the notebook, so you can always run everything top to bottom.

```
# Make the practicum package importable no matter where Jupyter was started.
import sys
from pathlib import Path

ROOT = Path.cwd()
while not (ROOT / "checks.py").exists() and ROOT != ROOT.parent:
    ROOT = ROOT.parent
sys.path.insert(0, str(ROOT))

import checks

DATA = ROOT / "data"
print("practicum root:", ROOT)
print("data folder   :", DATA, "|", len(list(DATA.glob('*'))), "files")
```

1. Verifying the environment (DEMO)

This is Worked Example 1.1 from the book. Run it first; if it fails, nothing else in the course will work.

The version block matters: whenever you ask for help with an error, paste this output with your question.

```

import platform
import numpy as np
import pandas as pd
import scipy
import sklearn
import matplotlib

print(f"Python      {sys.version.split()[0]} on {platform.system()}")
for lib in (np, pd, scipy, sklearn, matplotlib):
    print(f"{lib.__name__:<12} {lib.__version__}")

# Optional extras. The notebooks fall back gracefully if these are absent.
for name in ("seaborn", "statsmodels"):
    try:
        mod = __import__(name)
        print(f"{name:<12} {mod.__version__}")
    except ImportError:
        print(f"{name:<12} not installed (optional)")

```

```

# A five-line end-to-end smoke test: load, split, fit, score.
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

X, y = load_iris(return_X_y=True, as_frame=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)
acc = LogisticRegression(max_iter=1000).fit(X_tr, y_tr).score(X_te, y_te)
print(f"Smoke test accuracy: {acc:.4f}")

```

TASK 1.1

Assign the smoke-test accuracy you just obtained to `akurasi` and check it.

```

akurasi = None          # <- replace with the number printed above

checks.task("1.1", akurasi)

```

2. Reproducibility (DEMO)

Rule 1 of the course: **seed every source of randomness**, and use the modern generator API rather than the legacy global one.

`np.random.seed` mutates hidden global state that any library can also change. `np.random.default_rng(seed)` gives you an object you pass around explicitly, so two parts of a notebook cannot silently interfere with each other.

```
# Legacy API: hidden global state.
np.random.seed(0)
a = np.random.normal(size=3)
np.random.seed(0)
b = np.random.normal(size=3)
print("legacy, same seed twice :", np.allclose(a, b))

# Modern API: explicit, isolated state.
rng1 = np.random.default_rng(0)
rng2 = np.random.default_rng(0)
print("modern, two generators  :", np.allclose(rng1.normal(size=3),
                                              rng2.normal(size=3)))

# The trap the modern API removes: a library call in between changes your draw.
np.random.seed(0)
c = np.random.normal(size=3)
np.random.seed(0)
_ = np.random.uniform(size=1)      # some other code, somewhere else
d = np.random.normal(size=3)
print("legacy, interfered with :", np.allclose(c, d), " <- silent, and wrong")
```

3. First contact with data (DEMO)

`sales_clean.csv` is a tidy retail extract: one row per order. We will spend week 2 producing this file from a far messier one; today we just look at it.

```
sales = pd.read_csv(DATA / "sales_clean.csv", parse_dates=["order_date"])
print(sales.shape)
sales.head()
```

```
# The profiling helper used throughout this course (Chapter 2, section 2.5).
def profile(df):
    """Compact per-column profile: dtype, missingness, cardinality, an example."""
    out = pd.DataFrame({
        "dtype": df.dtypes.astype(str),
        "n_missing": df.isna().sum(),
        "pct_missing": (df.isna().mean() * 100).round(1),
        "n_unique": df.nunique(dropna=True),
        "example": [df[c].dropna().iloc[0] if df[c].notna().any() else None
                    for c in df.columns],
    })
    return out.sort_values("pct_missing", ascending=False)

profile(sales)
```

Three things to look for in any profile, before anything else:

- `n_unique == len(df)` → an **identifier**. Never a feature.
- `n_unique == 1` → a **constant**. Carries no information.
- a numeric minimum of `-999` or maximum of `9999` → an undeclared **missing-value sentinel** masquerading as data.

```
n = len(sales)
for col in sales.columns:
    u = sales[col].nunique(dropna=True)
    flag = ("  <- identifier" if u == n else
           "  <- constant" if u == 1 else "")
    print(f"{col:<14} {u:>6,} unique{flag}")
```

TASK 1.2

A university can offer intensive tutoring to 200 students out of 4,000 per semester and wants to know *which* 200 are most at risk of dropping out.

Is that primarily an **inference** problem or a **prediction** problem? (Chapter 1, section 1.1.2 — and think about what the output has to be.)

```
jenis_masalah = None    # "inference" or "prediction"

checks.task("1.2", jenis_masalah)
```

4. The base-rate trap (DEMO)

A colleague reports a classifier that detects equipment failure with **99.4% accuracy** in a factory where 0.5% of readings precede a failure.

Before congratulating them, compute the accuracy of the laziest possible model: the one that always predicts "no failure".

```
rng = np.random.default_rng(1)
n_readings = 200_000
failure = (rng.random(n_readings) < 0.005).astype(int)

always_no = np.zeros_like(failure)
accuracy_lazy = (always_no == failure).mean()

print(f"failures in the data : {failure.mean():.3%}")
print(f"'always no' accuracy : {accuracy_lazy:.3%}")
print(f"colleague's accuracy : 99.400%")
print()
print("The model that has learned nothing at all beats the reported model.")
```

This is the same arithmetic as the rare-disease screening example in Chapter 4, and it is why this course insists on the confusion matrix, on precision and recall, and on cost-weighted thresholds instead of accuracy.

TASK 1.3

State the accuracy (in **percent**) of the "always predict no failure" model.

```
akurasi_malas = None    # e.g. 88.8

checks.task("1.3", akurasi_malas)
```

5. Framing a vague request (TASK, written)

This is CRISP-DM phase 1, and it is the step that most often decides whether a project succeeds. Work through the six framing steps of Chapter 1, section 1.7 for the request below.

"Can you use data science to reduce customer churn?"

Fill in the markdown cell that follows. Be specific: an answer that would apply equally to any company has not done the work.

Your answer

1. **The decision.** Which decision will be made differently? Who makes it, how often, and what is the capacity constraint? →
2. **The prediction target.** Define it as a variable, with a time origin and a horizon. What does your definition exclude? →
3. **The prediction time.** When must the score exist? Which otherwise attractive features does that rule out? →
4. **The data inventory.** Which tables, which fields, who owns them, and is each available at prediction time? →
5. **Success, defined twice.** One technical metric, one business outcome. What is the baseline you must beat? →
6. **The risks.** Name two. At least one must be mechanical — a feedback loop, leakage, or a selection effect — not just "bias". →

6. Project layout (DEMO)

The convention this course uses. The central rule is that `data/raw` is **immutable**: every transformation is code, so `data/processed` can always be rebuilt from `data/raw` plus the repository.

```
layout = """
project/
├─ data/
│   ├─ raw/           # never edited, never committed if large or sensitive
│   ├─ interim/       # intermediate, regenerable
│   └─ processed/     # model-ready, regenerable
├─ notebooks/        # exploration; numbered, e.g. 01-eda.ipynb
├─ src/               # importable functions the notebooks call
├─ models/           # serialised fitted models
├─ reports/figures/  # exported graphics
├─ environment.yml    # the environment specification
└─ README.md         # what this is, how to run it
"""
print(layout)
```

7. Take-home

1. Re-run this notebook from a **fresh kernel** (Kernel → Restart Kernel and Run All Cells). If any cell fails, fix it now — every later week builds on this environment.
2. Export your environment specification and keep it with your project:

```
conda env export --from-history > environment.yml
```

Explain in one sentence why `--from-history` is preferred over a plain `conda env export`.

3. Complete the six framing steps in section 5 above.

Next week: we take `data/sales_raw.csv` — the file this week's tidy `sales_clean.csv` was made from — and clean it ourselves.